The Islamia University of Bahawalpur
Department of Information Technology

INFT-4041 · BSIT · 5th Semester · Fall 2026

E-Commerce Application Development

Chapter

20

Deployment, Maintenance and Performance

Part V — Running and Extending the Shop

Taught in	Week 15
Laboratory	Load test with cache on and off; crawlability measured
Assessment	**Assignment 2 due**

This is a class extract. It is *Chapter 20* of the unified handout, printed on its own so that a week's reading is a week's reading. Page numbers restart at 1 and are local to this extract; chapter, figure and table numbers are the ones the full volume gives them, so a reference you follow here lands in the same place there.

Cross-references to other chapters print as their numbers in the full volume. For the whole course in one file — front matter, the HEC tracing table, all twenty-three chapters and all seven appendices — see `ECAD_Fall2026_Handout.pdf`.

Dr. Mustafa Hameed · Department of Information Technology

Generated from `parts/` by `makechapters.py`; do not edit this PDF's source by hand

Chapter 20

Deployment, Maintenance and Performance

Learning Outcomes

By the end of this chapter you will be able to:

- **Measure** a shop's throughput and latency together, and **locate** its saturation point.
- **Explain** why throughput can fall as concurrency rises, and **say** what that means for a sale day.
- **Quantify** what the compiler cache is worth, and **order** optimisations by measured effect.
- **Distinguish** the cost of a backup from the cost of a restore, and **state** which one matters.
- **Assemble** a deployment checklist from this course's findings, with nothing on it that was not measured.

Before You Start

Chapter 5: images, containers, volumes and the Compose file. Chapter 16: the hardening file and what it closed. Chapter 14: reading a distribution rather than an average.

Key Terms in This Chapter

throughput • latency • concurrency • saturation point • queueing • opcode cache • reverse proxy • backup • restore • recovery time • rollback • end of life • pinning • HSTS

20.1 Two Numbers, and Why One of Them Lies

"How fast is the shop" has two answers that move in opposite directions, and quoting either alone is how capacity is mis-planned.

Throughput, Latency, Saturation

Throughput is requests served per second. It is a property of the server, and it is the number in the sales brochure.

Latency is how long one request took. It is what the customer experiences, and it is the number in the complaint.

The **saturation point** is the concurrency at which throughput stops rising. Below it, extra customers are served from spare capacity. Above it, extra customers join a queue — so latency rises steeply while throughput stays flat or *falls*.

Measured: The same page, requested by more people at once

From `code/m20_deploy.py -load` against the stack in Appendix A. Sixty requests at each level, spread across that many concurrent clients.

Clients	req/sec	Median	p95	Worst	Failed
1	11.0	81 ms	147 ms	158 ms	0
5	**42.1**	95 ms	181 ms	258 ms	0

20	**38.7**	**455 ms**	667 ms	814 ms	0

Throughput peaked at five clients and then fell. Twenty concurrent customers were served at 38.7 requests per second — *fewer* than five were. More customers, less work done.

And the median response grew 5.6 times, from 81 ms to 455 ms. Every one of those twenty customers waited half a second for a page that one customer alone gets in a twelfth of that.

Nothing failed. No error, no timeout, no log entry. The shop was past its saturation point and the only observable was that it had become slow — which is the same observable as a badly written query, a slow network or a customer's own machine. Load does not announce itself.

The saturation point is the number to know before a sale day. It is five here, on a laptop, serving an uncached home page. What matters is not the figure but that there *is* one, that it is measurable in ten minutes, and that past it every additional visitor makes the shop worse for everybody already in it.

20.2 The Cheapest Optimisation Is Already a Setting

Before any caching layer, any reverse proxy and any rewriting of queries, there is one line of configuration worth measuring.

Measured: The same load, with the compiler cache off and on

From `code/m20_deploy.py -opcache`. PHP compiles source to bytecode; the opcode cache keeps the result, so the files are compiled once rather than on every request.

Clients	req/sec off	req/sec on	Gain	Median off → on
1	8.1	10.1	1.3×	109 → 82 ms
5	**16.8**	**34.0**	**2.0**×	245 → 131 ms
20	21.7	30.3	1.4×	715 → 497 ms

Twice the throughput at five clients, for one line of configuration. No code was changed, no caching layer added, no query rewritten.

The gain is largest in the middle, and that is not a coincidence. With one client there is spare processor to waste, so the compilation is almost free. At twenty, the queue dominates and the saving is diluted. At five — the saturation point — the processor is the constraint, and removing work from it is worth exactly what it costs.

Check this before optimising anything else. A shop running without it is being benchmarked as a compiler rather than as a shop, and every conclusion drawn from such a measurement is about the wrong program. Chapter 5's Dockerfile installs it for this reason, and said so at the time.

The order to work in, from this course's own measurements

1. **Confirm the compiler cache is on.** 2×, one line.
2. **Minify the stylesheet** (Chapter 8). 240 KB off every page, one file, no visible change.
3. **Index what the pages actually sort by** (Chapter 6) — and re-measure, because at 50,000 products the indexes made it *slower*.

4. **Cache whole pages** for visitors who are not logged in. The largest win available and the one that introduces staleness, so it comes after the three that do not.
5. **Add servers.** Last, because everything above is cheaper, and because Chapter 9 showed that file-based sessions stop working the moment there are two.

20.3 Backups, and the Number Nobody Measures

Measured: What a backup costs, and what a restore costs

From `code/m20_deploy.py -backup`.

Database dump	**1.0 s**	572 KB
Database restore	**6.4 s**	
Shop files	0.8 s	52 MB in 4,024 files

The restore takes six times as long as the backup, and it is the only one of the two numbers that matters — because it is the one you will be timing while the shop is down and somebody is standing behind you.

Those figures are for a demonstration catalogue. At Chapter 6's 50,000 products the dump was a different order of magnitude, and the restore grew with it. **Measure your own, on your own data, and write the number down**: it is your recovery time, and it is what you will be asked for.

A backup nobody has restored is a file, not a backup. The only way to know a backup works is to restore it somewhere and look at the shop. Do it on a schedule, not after an incident.

Three things in the platform's own deployment advice that are wrong

Recorded because they are the first things a student will find, and all three are dated rather than debatable.

- **Its own `Dockerfile` does not build**, because the one in the release contains the line `docker-php-ext-install zip && && docker-php-ext-enable zip` — a shell syntax error. It has therefore never worked for anybody.
- **Its own compose file pins `mysql:5.7`**, which reached end of life in October 2023 and takes no further security fixes. This course's stack uses MariaDB 11 and Chapter 5 said why.
- **Its security guidance says to rename `.htaccess.txt`**, which Chapter 16 measured as doing nothing at all on a modern Apache.

Upstream artefacts are evidence, not authority. Read them to learn what the authors intended, then check whether they work.

20.4 Going Live

The checklist, with a measurement behind every line

Nothing here is advice this handout has not demonstrated.

1. **Change all three default passwords** — admin, database user, database root — and raise the minimum password length from six (Chapter 15, Chapter 16).
2. **Apply the hardening configuration** and re-run the battery. It moved 5 of 15 to 14 of 15 (Chapter 16).

3. **Obtain a certificate, serve over HTTPS, and only then switch on HSTS.** Before the certificate it locks you out of your own shop.
4. **Move `system/storage/` out of the web root**, rather than relying on a configuration file somebody will edit (Chapter 16).
5. **Rename the admin directory** and the matching path in `admin/config.php`.
6. **Confirm the compiler cache is on** — 2×, measured above.
7. **Fix `config_checkout_payment_address`**, without which no guest can complete a checkout (Chapter 10).
8. **Populate `oc_api_ip`**, without which the admin panel cannot change an order (Chapter 18).
9. **Take a backup, restore it somewhere else, and time the restore.** Write the number down.
10. **Know your saturation point** before the first sale day.

Four of those ten are faults in the shipped software that every page of the shop will hide from you. That is the argument of the whole course in one list.

Common Pitfalls

- **Quoting throughput without latency.** At twenty clients this shop served 38.7 requests a second and every customer waited half a second.
- **Assuming more concurrency means more work done.** Past saturation it means less.
- **Benchmarking with the compiler cache off** and drawing conclusions about anything else.
- **Adding a caching layer before checking the free settings.**
- **Measuring the backup and never the restore.**
- **Never restoring a backup at all.**
- **Switching on HSTS before there is a certificate.**
- **Copying upstream's own Dockerfile or compose file.** One does not build and the other pins software that is out of support.
- **Deploying with the course's teaching passwords.**

Try It Yourself: Find where your shop falls over, then make it fall over later

```
# --- 1. Predict your saturation point --------------------------------
# How many concurrent customers before throughput stops rising?
# Write a number down before measuring.
cd code
python m20_deploy.py --load

# --- 2. Read both columns together -----------------------------------
# At the level where req/sec peaks, what is the median? And at
# twice that concurrency? Write one sentence a shop owner would
# understand.

# --- 3. Measure the free optimisation --------------------------------
python m20_deploy.py --opcache
# Explain why the gain is biggest in the MIDDLE row.

# --- 4. Apply Chapter 8's finding and re-measure ---------------------
# Minify bootstrap.css, then run --load again. Did req/sec move?
# It should not, by much. Say why not, and what DID improve.
```

```
# --- 5. Time a restore ------------------------------------------------
python m20_deploy.py --backup

# --- 6. Now do it for real --------------------------------------------
# Seed 50,000 products (seed/catalogue.py), then time the dump and
# the restore again. Record both. That is your recovery time.
# Restore the demo catalogue afterwards.

# --- 7. Prove the backup works ----------------------------------------
# docker compose down -v, up -d, restore your dump, and open the
# shop. If the products are not there, you did not have a backup.

# --- 8. Write the go-live checklist -----------------------------------
# Ten lines, each with the measurement or chapter that justifies it.
# Anything you cannot justify, take off the list.
```

What to notice. Step 7 is the only step that proves anything. Everyone takes backups; the question that separates a professional from an amateur is when they last restored one.

Checkpoint — Test Yourself

1. At 5 concurrent clients the shop served 42.1 requests per second; at 20 it served 38.7. Explain the mechanism, say what a shop owner should do with that information before a sale day, and give one change that would move the turning point.
2. The compiler cache doubled throughput at 5 clients but gave only 1.3× at 1 and 1.4× at 20. Explain both ends of that pattern, and say what it implies about when to benchmark.
3. Your client's backup runs nightly and has never been restored. Write the case for a restore test in terms your client will act on, state exactly what you would do and how long it would take, and say what you would measure and record.

Chapter Summary

- Throughput and latency are different questions and move in opposite directions. Quoting either alone mis-plans capacity.
- Measured: throughput peaked at 5 concurrent clients (42.1 req/sec) and *fell* to 38.7 at 20, while the median response grew 5.6× from 81 ms to 455 ms. Nothing failed and nothing was logged.
- The saturation point is measurable in ten minutes and is the number to know before a sale day.
- Measured: the opcode cache doubled throughput at the saturation point for one line of configuration, and the gain was largest in the middle — spare capacity at one client, queueing at twenty.
- Optimise in measured order: compiler cache, then assets, then indexes (re-measuring, because at 50,000 products they hurt), then page caching, then servers.
- Measured: a 1.0 s backup took 6.4 s to restore. The restore is the number that matters, and a backup nobody has restored is a file.
- Upstream's own Dockerfile does not build, its compose file pins software out of support, and its security advice does nothing on a modern Apache. Artefacts are evidence, not authority.
- Four of the ten go-live checklist items are faults in the shipped software that every page of the shop hides from you.

Review Questions

1. **(MCQ)** Going from 5 to 20 concurrent clients changed throughput from 42.1 to: (a) 168 req/sec; (b) 84 req/sec; (c) 38.7 req/sec; (d) it failed.
2. **(MCQ)** The opcode cache gave its largest gain at: (a) 1 client; (b) 5 clients; (c) 20 clients; (d) the same at all three.
3. **(Short)** Define the saturation point and explain why throughput can fall beyond it.
4. **(Short)** Why is the restore time, rather than the backup time, the figure a business should be given?
5. **(Short)** Explain why benchmarking a PHP application with the opcode cache disabled produces conclusions about the wrong program.
6. **(Applied)** A shop expects 300 concurrent visitors for a two-hour sale. Its measured saturation point is 5 and its median response at 20 is 455 ms. Set out what you would do, in order, with an expected effect for each step and a statement of which steps you could verify beforehand and which you could not.
7. **(Applied)** Design the backup and recovery procedure for a shop taking 200 orders a day. State what is backed up, how often, where it is kept, how long you may lose, how long a restore takes, who performs it, how the procedure is tested, and how you would discover that a night's backup had silently failed.